Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

A. Bảng thuật ngữ 2

PHỤ LỤC A

Bảng thuật ngữ

Thuật ngữ kỹ thuật dùng trong sách, xếp theo bảng chữ cái. Cột đầu ghi thuật ngữ tiếng Anh; nếu sách dùng dạng tiếng Việt, tên tiếng Việt ghi trong ngoặc. Cột "Chương" ghi chương trình bày chính của thuật ngữ. Trong các chương, thuật ngữ được chú giải ở lần xuất hiện đầu tiên; các thuật ngữ chuẩn quốc tế (API Gateway, microservices, endpoint...) giữ tiếng Anh.

Thuật ngữ	Giải thích ngắn	Chương
Accepted Risk (rủi ro được chấp nhận)	Rủi ro đã được nhận diện và tạm thời chấp nhận kèm điều kiện kiểm soát; phải ghi rõ và không được coi là kiến trúc mục tiêu.	Ch.8
Actor / Subject / Delegation	Trong phân quyền (authorization): subject là người dùng được đại diện; actor là workload đang gọi; delegation là quyền để actor hành động thay subject. Ba phần phải tách biệt để tránh confused deputy.	Ch.9
Aggregate	Cụm entity/value object được xem như một đơn vị nhất quán, truy cập qua một "gốc" (aggregate root) để bảo vệ ràng buộc nghiệp vụ.	Ch.2
Alert Fatigue (mệt mỏi vì cảnh báo)	Tình trạng đội vận hành nhận quá nhiều cảnh báo giả hoặc kém quan trọng đến mức bỏ qua cả sự cố thật; khắc phục bằng cảnh báo theo SLO và gom nhóm cảnh báo.	Ch.11
Anti-Corruption Layer	ACL — lớp dịch trung gian giúp một service không bị "nhiễm" mô hình dữ liệu của hệ thống ngoài/legacy khi tích hợp.	Ch.10
API Composition	Gộp dữ liệu từ nhiều service bằng cách gọi các service sở hữu dữ liệu rồi ghép kết quả ở tầng gọi (thay cho join trong database).	Ch.7
API Gateway	Lớp xử lý request tại biên, tập trung các nghiệp vụ cắt ngang (cross-cutting concern) như routing, xác minh token ở biên và rate limiting; không nắm quyền phân quyền nghiệp vụ của từng service.	Ch.8
Backend for Frontend	BFF — một gateway riêng "may đo" cho từng loại client (web, mobile), trả về dữ liệu vừa vặn nhu cầu client đó.	Ch.8
BASE	Basically Available, Soft state, Eventually consistent — triết lý dữ liệu phân tán đối trọng với ACID: ưu tiên tính sẵn sàng, chấp nhận trạng thái trung gian và nhất quán cuối cùng.	Ch.6
Blue/Green Deployment	Chạy song song hai môi trường (cũ "blue" và mới "green"), chuyển traffic sang bản mới khi sẵn sàng và có thể chuyển lại nếu phiên bản dữ liệu/API vẫn tương thích.	Ch.12
Bottleneck (điểm nghẽn, nút thắt cổ chai)	Thành phần chậm nhất giới hạn hiệu năng của toàn hệ thống.	Ch.1

Thuật ngữ	Giải thích ngắn	Chương
Bounded Context	Phạm vi mà một mô hình domain, bộ thuật ngữ và quy tắc có hiệu lực. Đây là ranh giới mô hình; không mặc nhiên ánh xạ một-một sang một service triển khai độc lập.	Ch.2
Branch by Abstraction	Kỹ thuật chuyển đổi: đặt một lớp trừu tượng trước phần hiện thực cũ, xây phiên bản mới phía sau cùng interface (bật/tắt bằng feature flag), chuyển dần lưu lượng rồi gỡ bản cũ.	Ch.10
Build	Biên dịch mã nguồn và đóng gói thành artifact chạy được.	Ch.12
Bulkhead	Cô lập tài nguyên (thread pool, kết nối) theo nhóm để một phần quá tải không gây đình trệ phần còn lại. Ẩn dụ: khoang kín của tàu thủy.	Ch.4
Business Logic	Phần mã nguồn cốt lõi thực thi các quy tắc, tính toán và luồng nghiệp vụ đặc thù của miền.	Ch.1
Canary Deployment	Phát hành dần cho một nhóm nhỏ người dùng trước, theo dõi, rồi mới mở rộng toàn bộ.	Ch.12
CAP Theorem	Định lý CAP — khi xảy ra network partition (phân vùng mạng), hệ phân tán buộc phải chọn giữa Consistency (nhất quán) và Availability (sẵn sàng); không phải "chọn 2 trong 3" (xem Chương 7).	Ch.7
Cardinality Explosion	Số time-series của metric tăng bùng nổ khi gắn nhãn có giá trị biến thiên cao (userId, submissionId), làm quá tải hệ thống metric; chi tiết theo từng đối tượng nên để cho log/trace.	Ch.11
Change Data Capture	CDC — bắt các thay đổi trong database (insert/update/delete) thành luồng sự kiện cho hệ thống khác tiêu thụ.	Ch.10
Chaos Engineering	Chủ động gây lỗi có kiểm soát trên hệ thống thật để kiểm chứng khả năng chịu lỗi.	Ch.11
Choreography	Cách phối hợp saga không có "nhạc trưởng": mỗi service tự phản ứng với event và phát event tiếp theo.	Ch.6
CI/CD	Continuous Integration / Continuous Delivery — tự động tích hợp, build, test và chuẩn bị bản sẵn sàng phát hành; triển khai production có thể cần phê duyệt. Continuous Deployment thì tự động triển khai mọi thay đổi vượt qua các cổng kiểm tra.	Ch.12
Circuit Breaker	Cơ chế "ngắt mạch" tạm dừng gọi tới một service đang lỗi để tránh lan truyền lỗi và giúp service đó có thời gian hồi phục. Ẩn dụ: cầu dao điện.	Ch.4
Cluster	Nhóm nhiều máy/node hoạt động như một hệ thống thống nhất.	Ch.12
Compensating Transaction (giao dịch bù trừ)	Thao tác "làm ngược lại" để hoàn tác hệ quả của một bước đã thực hiện trong saga (vì không thể rollback như giao dịch database).	Ch.6
Configuration Drift (sai lệch cấu hình)	Cấu hình các máy chủ/môi trường dần khác nhau do chỉnh sửa thủ công, khiến môi trường không tái tạo được; IaC và hạ tầng bất biến là cách phòng tránh.	Ch.12
Confused Deputy	Lỗ hổng khi service A gọi service B thay mặt người dùng nhưng B không phân biệt được A đang dùng quyền của người dùng hay đặc quyền riêng của A, dẫn đến leo thang đặc quyền.	Ch.9
Consistency Window	Khoảng thời gian từ lúc một sự kiện xảy ra cho đến khi mọi dịch vụ liên quan phản ánh trạng thái mới đó.	Ch.6
Consumer-Driven Contract Testing	CDCT — consumer công bố kỳ vọng về request/response, provider xác minh contract đó trong CI trước khi deploy (ví dụ: Pact). Sách dành `CDC` cho Change Data Capture.	Ch.3
Container	Gói đóng kín ứng dụng cùng mọi phụ thuộc để chạy giống nhau ở mọi môi trường.	Ch.12
Context Map	Bản đồ quan hệ giữa các Bounded Context, ghi rõ kiểu tích hợp giữa từng cặp.	Ch.2
Contract Testing (kiểm thử hợp đồng)	Kiểm thử xác nhận hai service tuân thủ đúng "hợp đồng" API đã thỏa thuận, không cần chạy cả hai cùng lúc.	Ch.3
Conway's Law	Cấu trúc hệ thống có xu hướng phản chiếu cấu trúc giao tiếp của tổ chức tạo ra nó.	Ch.2
Core Domain	Subdomain tạo khác biệt chiến lược và đáng nhận đầu tư thiết kế cao nhất; không phải tên gọi chung cho mọi Bounded Context nghiệp vụ.	Ch.2

Thuật ngữ	Giải thích ngắn	Chương
Correlation ID	Mã định danh gom nhóm một tiến trình nghiệp vụ qua nhiều request, trace và thông điệp bất đồng bộ (ví dụ trường `correlationId` trong event envelope); phân biệt với `requestId` (`X-Request-ID` cho từng HTTP request ở biên) và `traceId` (`traceparent` của W3C Trace Context).	Ch.5, 11
CQRS	Command Query Responsibility Segregation — tách mô hình ghi (command) và mô hình đọc (query) thành hai đường riêng để tối ưu độc lập.	Ch.7
Credential (thông tin xác thực)	Dữ liệu chứng minh danh tính khi xác thực, như mật khẩu, token, khóa API hoặc chứng chỉ.	Ch.9
Cross-cutting Concern (nghiệp vụ cắt ngang)	Các nghiệp vụ cắt ngang toàn bộ hệ thống (như bảo mật, logging, rate limiting) không thuộc về một domain cụ thể nào.	Ch.8
DB Queue	Dùng một bảng database làm hàng đợi công việc — đơn giản hơn message broker cho nhu cầu nhỏ.	Ch.5
Dead Letter Topic/Queue	DLT — nơi chứa message xử lý thất bại sau nhiều lần thử, để xử lý riêng thay vì retry vô hạn.	Ch.5
Debug	Tìm và sửa nguyên nhân lỗi trong phần mềm.	Ch.1
Decision Matrix (ma trận quyết định)	Bảng đối chiếu tiêu chí để ra quyết định kiến trúc, ví dụ khi nào nên chuyển sang microservices hoặc ưu tiên công việc theo tác động × nỗ lực.	Ch.10
Defense in Depth (phòng thủ chiều sâu)	Bảo vệ nhiều lớp nối tiếp (mạng, ingress, service, dữ liệu) để khi một lớp bị vượt qua, các lớp còn lại vẫn đứng vững.	Ch.9
Distributed Monolith	Anti-pattern: đã có ranh giới service về hình thức nhưng dữ liệu, thư viện hoặc việc triển khai vẫn ràng buộc chặt, buộc phải thay đổi đồng loạt (lockstep), nên các service không thể tiến hóa độc lập.	Ch.1
Distributed Tracing (truy vết phân tán)	Theo dõi một request xuyên qua nhiều service bằng trace/span có đo thời gian từng bước — trả lời câu hỏi “chậm ở đâu” trong hệ phân tán.	Ch.11
Docker Compose	Công cụ định nghĩa và chạy nhiều container cùng lúc bằng một file YAML.	Ch.12
Domain-Driven Design	DDD — phương pháp thiết kế lấy mô hình nghiệp vụ (domain) làm trung tâm, dùng ngôn ngữ chung giữa lập trình viên và chuyên gia nghiệp vụ.	Ch.2
Downtime	Khoảng thời gian hệ thống không phục vụ được.	Ch.4
DTO	Data Transfer Object — đối tượng chỉ để truyền dữ liệu qua ranh giới (API), tách biệt với entity nội bộ.	Ch.3
Dual-write (ghi kép)	Ghi vào database rồi phát sự kiện bằng hai thao tác tách rời; nếu một bước thất bại, dữ liệu và sự kiện không còn nhất quán. Outbox Pattern là cách khắc phục.	Ch.10
Effectively-once	Đảm bảo “như thể xử lý đúng một lần” bằng at-least-once + idempotency (vì exactly-once tuyệt đối xuyên suốt các hệ thống là bất khả thi).	Ch.5
Endpoint	Điểm cuối giao tiếp mạng (thường là một URL cụ thể) nơi API tiếp nhận và trả về dữ liệu cho client.	Ch.3
Error Budget (ngân sách lỗi)	Phần chênh lệch cho phép `100% - SLO` trong một kỳ đo; còn ngân sách thì ưu tiên tính năng mới, cạn thì ưu tiên độ tin cậy.	Ch.4, 11
Event Sourcing	Lưu trạng thái dưới dạng chuỗi sự kiện bất biến (thay vì chỉ trạng thái hiện tại); trạng thái được tái dựng bằng cách “phát lại” sự kiện. Ẩn dụ: sổ sao kê ngân hàng.	Ch.7
Event Storming	Workshop dùng sticky note để cùng khám phá domain qua các sự kiện nghiệp vụ.	Ch.2
Event-driven (hướng sự kiện)	Kiến trúc trong đó các service giao tiếp gián tiếp bằng cách phát và nghe sự kiện.	Ch.5
Eventual Consistency	Dữ liệu giữa các service cuối cùng sẽ đồng bộ, nhưng có một khoảng trễ được chấp nhận — không có cận trên cam kết (đối lập với strong consistency).	Ch.6

Thuật ngữ	Giải thích ngắn	Chương
Expand/Contract	Kỹ thuật cập nhật schema tương thích ngược theo ba bước: mở rộng (thêm cột/bảng mới), chuyển dữ liệu, rồi thu hẹp (gỡ phần cũ) khi phiên bản code cũ đã ngừng chạy.	Ch.12
Flyway / Liquibase	Công cụ quản lý phiên bản schema database (versioned migrations), chạy như một bước riêng trước khi deploy service.	Ch.12
GitOps	Quản lý hạ tầng/triển khai bằng Git làm nguồn sự thật duy nhất; mọi thay đổi qua commit/PR.	Ch.12
GraphQL	Ngôn ngữ truy vấn API cho phép client yêu cầu đúng dữ liệu cần, tránh over/under-fetching.	Ch.3
gRPC	Framework RPC hiệu năng cao của Google chạy trên HTTP/2 + Protobuf, hợp cho giao tiếp nội bộ service-to-service.	Ch.4
HATEOAS	Mức "trưởng thành" của REST trong đó response kèm các link điều hướng hành động tiếp theo.	Ch.3
Head-of-Line Blocking	Một message xử lý chậm ở đầu partition chặn mọi message phía sau trong cùng partition.	Ch.5
Header	Tiêu đề của một gói tin hoặc yêu cầu HTTP, dùng để truyền tải các siêu dữ liệu (metadata) như access token hoặc mã định danh yêu cầu (`X-Request-ID`).	Ch.8
Health Check	Endpoint (thường là `/health`) cho biết service còn chạy và sẵn sàng nhận request; được registry, load balancer và orchestrator dùng để định tuyến và tự phục hồi. Gồm liveness, readiness và startup.	Ch.11
Hexagonal Architecture	Ports & Adapters — tách lõi nghiệp vụ khỏi hạ tầng qua "cổng" và "bộ chuyển đổi", giúp dễ test và thay công nghệ.	Ch.3
Idempotency (tính lũy đẳng)	Tính chất của một thao tác: thực hiện nhiều lần cho kết quả giống như thực hiện một lần duy nhất. Ẩn dụ: bấm nút gọi thang máy 5 lần, thang vẫn chỉ đến một lần.	Ch.3, 5
Immutable Infrastructure (hạ tầng bất biến)	Không sửa trực tiếp máy chủ đang chạy; mọi thay đổi được thực hiện bằng cách tạo phiên bản mới và thay thế phiên bản cũ.	Ch.12
Implementation Drift (sự trôi lệch triển khai)	Các service hiện thực cùng một chính sách theo những cách khác nhau (thư viện, phiên bản, quy tắc kiểm tra), dẫn đến hành vi không nhất quán dù cùng tên gọi.	Ch.8
Independent Silo Monoliths	Nhiều ứng dụng monolith tách rời, mỗi ứng dụng có codebase/deployment riêng nhưng thiếu kiến trúc và contract thống nhất; khác Distributed Monolith.	Ch.1
Infrastructure as Code	IaC — mô tả hạ tầng bằng mã/khai báo để tạo lại tự động và nhất quán.	Ch.12
JWKS	JSON Web Key Set — tập khóa công khai (public key) do issuer công bố để resource server xác minh token và tự làm mới khóa khi luân phiên khóa (rotation).	Ch.9
JWT	JSON Web Token — định dạng token; sách chủ yếu dùng compact JWS có chữ ký để resource server xác minh claim mà không cần tra database ở mỗi request.	Ch.9
k3s	Bản Kubernetes gọn nhẹ, hợp cho edge và môi trường tài nguyên thấp.	Ch.12
Load Balancing (cân bằng tải)	Phân phối request đến nhiều instance để tránh quá tải một instance.	Ch.4
Message Broker	Trung gian nhận/lưu/chuyển message giữa producer và consumer, cho phép giao tiếp bất đồng bộ. Ẩn dụ: bưu điện giữ thư hộ.	Ch.5
Microservice	Dịch vụ nhỏ, tự chủ, triển khai độc lập, sở hữu dữ liệu riêng và giao tiếp với các dịch vụ khác qua mạng.	Ch.1
Migration (chuyển đổi) / Migration Path (lộ trình chuyển đổi)	Quá trình đưa một hệ thống đang vận hành từ kiến trúc hiện tại sang kiến trúc mục tiêu; lộ trình chuyển đổi là chuỗi bước tăng dần, mỗi bước mang lại giá trị và có thể hoàn tác.	Ch.10
Modular Monolith	Ứng dụng nguyên khối nhưng được chia thành các module có ranh giới rõ ràng bên trong — một lựa chọn hợp lệ trước khi tách microservices.	Ch.1
Monolith (nguyên khối)	Kiến trúc nguyên khối: ứng dụng được đóng gói và triển khai như một khối duy nhất.	Ch.1

Thuật ngữ	Giải thích ngắn	Chương
mTLS	Mutual TLS — hai phía xác thực certificate/workload identity và thiết lập kênh mã hóa. mTLS không tự chứng minh nội dung claim của người dùng hoặc thay cho phân quyền nghiệp vụ.	Ch.9
MTTF / MTBF / MTTR	Thời gian trung bình tới khi hỏng, giữa hai lần hỏng, và để khắc phục; `MTBF = MTTF + MTTR`.	Ch.4
OAuth2	Chuẩn ủy quyền cho phép cấp quyền truy cập tài nguyên mà không chia sẻ mật khẩu.	Ch.9
Observability	Khả năng hiểu trạng thái bên trong hệ thống từ dữ liệu nó phát ra (log, metric, trace) — đủ để đặt câu hỏi mới mà không cần triển khai lại.	Ch.11
Observability Maturity Model (mô hình trưởng thành)	Thang 5 cấp (Reactive → Predictive) giúp xác định năng lực observability hiện tại và mục tiêu phù hợp; khác với Richardson Maturity Model của REST.	Ch.11
OIDC	OpenID Connect — lớp xác thực (authentication) xây trên OAuth2, bổ sung danh tính người dùng qua ID token.	Ch.9
OpenAPI	OpenAPI / Swagger — chuẩn mô tả REST API dạng máy đọc được, dùng để sinh tài liệu và code.	Ch.3
OpenFeign	Thư viện Spring cho phép gọi REST service khác qua khai báo interface (declarative HTTP client).	Ch.4
Orchestration	Cách phối hợp saga có "nhạc trưởng" trung tâm điều khiển từng bước.	Ch.6
Outbox Pattern	Ghi sự kiện vào một bảng "outbox" cùng transaction với dữ liệu nghiệp vụ, rồi phát sang message broker ở bước riêng — tránh mất sự kiện do ghi kép (dual-write); relay có thể phát trùng nên consumer vẫn cần idempotent.	Ch.10
Overhead	Chi phí tài nguyên/độ trễ phát sinh thêm ngoài công việc chính.	Ch.1
Pact	Công cụ contract testing theo hướng consumer-driven.	Ch.3
Parallel Run	Chạy song song logic cũ và mới trên dữ liệu thật, chỉ trả kết quả của bản cũ và so sánh để phát hiện sai lệch trước khi chuyển hẳn sang bản mới.	Ch.10
Pipeline	Chuỗi bước tự động (build, test, deploy…) nối tiếp nhau.	Ch.12
Pivot Transaction (giao dịch bản lề)	Bước trong saga mà sau đó không thể quay đầu: các bước trước có thể bù trừ, các bước sau phải thử lại đến khi thành công.	Ch.6
PKCE	Proof Key for Code Exchange — cơ chế bảo vệ luồng OAuth2 Authorization Code cho client công khai (mobile/SPA) chống đánh cắp mã.	Ch.9
Poison Pill	Message lỗi/không xử lý được làm consumer kẹt vòng lặp retry; cần tách sang DLT.	Ch.5
Polyglot Persistence	Mỗi service tự chọn loại database phù hợp nhất với nhu cầu của nó (SQL, NoSQL, cache…).	Ch.7
Rate Limiting	Giới hạn số request trong một khoảng thời gian để bảo vệ hệ thống khỏi quá tải/lạm dụng.	Ch.8
RBAC	Role-Based Access Control — phân quyền theo vai trò được gán cho người dùng.	Ch.9
Refactor	Cải thiện cấu trúc mã mà không thay đổi hành vi.	Ch.1
Resilience4j	Thư viện Java hiện thực các resilience pattern (Circuit Breaker, Retry, Timeout, Bulkhead) qua annotation/config.	Ch.4
Resource Server	Service/API sở hữu tài nguyên được bảo vệ; nó xác minh access token dành cho mình và tự phân quyền cục bộ (local authorization).	Ch.9
REST	Kiểu kiến trúc API dựa trên resource và HTTP verbs (GET/POST/PUT/DELETE).	Ch.3
Retry (thử lại)	Thử lại request khi gặp lỗi tạm thời.	Ch.4
Rolling Update	Thay lần lượt từng instance từ phiên bản cũ sang phiên bản mới mà không cần gấp đôi tài nguyên; trong quá trình này cả hai phiên bản cùng chạy.	Ch.12
Routing (định tuyến)	Quá trình định tuyến, điều hướng request từ nguồn đến đúng đích dựa trên các quy tắc (path, header).	Ch.8

Thuật ngữ	Giải thích ngắn	Chương
Saga Pattern	Phối hợp chuỗi giao dịch cục bộ (local transaction); khi bước sau thất bại, hành động bù trừ (compensating action) đưa nghiệp vụ về trạng thái chấp nhận được, không nhất thiết khôi phục bit-for-bit như rollback database.	Ch.6
Scaling (mở rộng)	Mở rộng khả năng phục vụ bằng cách thêm tài nguyên (scale up) hoặc thêm instance (scale out).	Ch.1
Semantic Lock	Cờ trạng thái ở tầng ứng dụng đánh dấu bản ghi đang được một saga xử lý, để các saga khác không can thiệp.	Ch.6
Service	Một thành phần phần mềm tự chủ cung cấp một nhóm chức năng qua giao diện rõ ràng.	Ch.1
Service Discovery	Cơ chế để service tìm địa chỉ động của service khác mà không hard-code IP.	Ch.4
Service Mesh	Lớp hạ tầng (thường qua sidecar) lo giao tiếp service-to-service: routing, mTLS, retry, observability.	Ch.12
Service Registry	Danh bạ trung tâm lưu địa chỉ các instance đang hoạt động; service tự đăng ký (register) và bên gọi tra cứu để định tuyến (ví dụ: Netflix Eureka).	Ch.4
Shadow Deployment	Nhân bản lưu lượng thật sang phiên bản mới nhưng bỏ kết quả của nó, để kiểm chứng dưới tải thật trước khi phát hành; mọi tác dụng phụ phải được cô lập.	Ch.12
Sidecar	Sidecar pattern — đặt một container phụ chạy cạnh service chính để đảm nhận các tác vụ chung (proxy, logging) mà không phải sửa đổi mã nguồn của service đó.	Ch.12
Signed Identity Propagation	Truyền token/metadata có chữ ký qua ranh giới tin cậy (trust boundary); service đích kiểm tra issuer, audience, signature và freshness trước khi dùng claim.	Ch.8, 9
SLI/SLO/SLA	Chỉ số đo chất lượng dịch vụ (SLI), mục tiêu nội bộ (SLO) và cam kết với khách hàng (SLA).	Ch.11
Strangler Fig	Strangler Fig pattern — thay thế dần hệ cũ bằng cách bọc và chuyển từng phần chức năng sang hệ mới, đến khi hệ thống cũ bị thay thế hoàn toàn.	Ch.10
Strong Consistency (nhất quán mạnh)	Mọi thao tác đọc luôn thấy lần ghi mới nhất, không có khoảng tạm thời đọc phải dữ liệu cũ (stale).	Ch.7
Structured Logging (log có cấu trúc)	Ghi log ở dạng có cấu trúc (thường là JSON) với các trường như timestamp, service, traceId để truy vấn, lọc và tổng hợp tự động.	Ch.11
Sysbox	Runtime container cho phép chạy Docker/Kubernetes lồng bên trong container an toàn hơn.	Ch.12
Team	Nhóm/đội phát triển.	Ch.1
Tiered Trust (tin cậy phân tầng)	Bốn tầng tin cậy (trust tier) tăng bằng chứng theo rủi ro: T0 request từ bên ngoài chưa tin cậy; T1 bằng chứng người dùng có chữ ký + phân quyền cục bộ; T2 thêm workload/actor/delegation; T3 bảo vệ job/command bất đồng bộ.	Ch.9
Timeout	Bên gọi ngừng chờ sau một thời hạn; công việc phía remote có thể vẫn tiếp tục, nên luồng nhạy cảm cần cancellation, idempotency hoặc fencing phù hợp.	Ch.4
Token Rotation (luân phiên token)	Mỗi lần làm mới cấp một refresh token mới và vô hiệu hóa token cũ, nhờ đó phát hiện được việc dùng lại token đã lộ.	Ch.9
Traffic	Lưu lượng truy cập mạng hoặc khối lượng dữ liệu trao đổi giữa các thành phần hệ thống.	Ch.8
Turnstile	Cloudflare Turnstile — dịch vụ challenge chống bot (thay thế CAPTCHA), bảo vệ các form nhạy cảm như đăng nhập/đăng ký/khôi phục mật khẩu.	Ch.9
Two-Phase Commit	2PC — giao thức commit phân tán hai pha (prepare rồi commit); đảm bảo nhất quán nhưng khóa tài nguyên và kém chịu lỗi, ít dùng trong microservices.	Ch.6

Thuật ngữ	Giải thích ngắn	Chương
Ubiquitous Language (ngôn ngữ chung)	Bộ thuật ngữ thống nhất giữa lập trình viên và chuyên gia nghiệp vụ trong một Bounded Context, dùng cả trong trao đổi lẫn trong mã.	Ch.2
Virtual Threads	Luồng nhẹ do JVM quản lý từ Java 21, giúp các lệnh gọi blocking I/O ít chiếm luồng nền tảng.	Ch.4
WebSocket / SSE	Hai cơ chế server đẩy dữ liệu real-time về client: WebSocket (hai chiều, full-duplex) và Server-Sent Events (một chiều server→client).	Ch.5
Workload Identity	Danh tính mật mã của service/job đang chạy, dùng để xác thực giữa các service (service-to-service); khác với user subject mà workload có thể đại diện.	Ch.9
Zero Trust	Không mặc nhiên tin cậy chỉ vì vị trí mạng; mỗi lần truy cập tài nguyên cần xác thực và phân quyền phù hợp với chính sách.	Ch.9